
UNIDAD 04

Condicionales

Programación en Computación · Ingeniería Electromecánica — 2.º año · UTN FR Reconquista
Longhi Pablo · Talijancic Iván

Qué vamos a ver

- 01 `if` — ejecutar **sólo si**
- 02 `if / else` — dos caminos
- 03 `else if` — varios casos, y por qué **el orden importa**
- 04 `switch-case` y el `break` que falta
- 05 Cuál de los dos conviene en cada situación

Nada de hoy es nuevo del todo. Las condiciones son las de la clase pasada. Lo que cambia es que ahora **deciden**.

PREGUNTA

La clase pasada calculamos esto. ¿Y después qué?

```
measured >= lowerLimit && measured <= upperLimit
```

PARTE 1

| Ejecutar sólo si

La forma mínima

src/app.js

```
const temperature = 91

if (temperature > 85) {
  console.log('Temperatura FUERA DE RANGO')
}

console.log('Fin del control')
```

SALIDA

```
Temperatura FUERA DE RANGO
Fin del control
```

Con 70 °C la primera línea **no se imprime**: el programa salta directo al final.

Que se lea como una oración

× CUESTA LEERLO

```
if (measured >= lower &&  
    measured <= upper) {  
  console.log('OK')  
}
```

✓ SE LEE SOLO

```
const isWithinRange =  
  measured >= lower &&  
  measured <= upper  
  
if (isWithinRange) {  
  console.log('OK')  
}
```

Guardá la condición en una variable con nombre. El `if` queda de una línea.

Las llaves van siempre

src/app.js

```
if (temperature > 85) console.log('Alarma')  
  console.log('Se registró el evento')
```

ERROR FRECUENTE

La segunda línea está **fuera** del `if`, por más que la indentación diga lo contrario. Se imprime **siempre**, aunque la temperatura esté bien.

JavaScript te deja omitirlas. **En la cátedra no.** Con llaves, el error no puede ocurrir.

PARTE 2

| Dos caminos

Uno u otro, siempre

src/app.js

```
if (measured > UPPER_LIMIT) {  
  console.log('Tensión ALTA')  
} else {  
  console.log('Dentro del límite')  
}
```

El `else` no lleva condición: es «en cualquier otro caso».

Con `if` / `else` es **imposible** que no se imprima nada. Uno de los dos se ejecuta sí o sí.

PARTE 3

| Varios casos

Clasificar una medición

src/app.js

```
const measured = 372.5

if (measured < 361) {
  console.log('BAJA')
} else if (measured <= 399) {
  console.log('NORMAL')
} else {
  console.log('ALTA')
}
```

SALIDA

NORMAL

Sólo se ejecuta UNA rama

Se evalúan de
arriba hacia abajo.

Apenas una da `true`, se ejecuta
y se sale.

Por eso la segunda condición es `measured <= 399` y no `measured >= 361 && measured <= 399`.

Si el programa llegó a la segunda rama, **ya sabemos**
que no es menor a 361: la primera dio `false`.

REPETIR ESA MITAD ES RUIDO.

PREGUNTA

Mismos rangos, orden
invertido. ¿Qué
imprime con 300 V?

```
if (measured <= 399) ... else if (measured < 361) ...
```

El orden importa muchísimo

× ASÍ NO

```
if (measured <= 399) {  
  console.log('NORMAL')  
} else if (measured < 361) {  
  console.log('BAJA')  
}
```

300 <= 399 da true : entra acá. La segunda **nunca se evalúa**.

✓ ASÍ SÍ

```
if (measured < 361) {  
  console.log('BAJA')  
} else if (measured <= 399) {  
  console.log('NORMAL')  
}
```

Rangos **de menor a mayor**, sin saltar.

Y lo peor es que no falla

No hay error. No hay aviso.

El programa clasifica mal **siempre**, y te enterás cuando alguien lee el informe.

ERROR FRECUENTE

La regla: de lo más específico a lo más general. Con rangos numéricos, **ordenados** de menor a mayor. Si los escribís en orden, el problema no puede aparecer.

Poné el `else` final

src/app.js

```
if (temperature > 85) {  
  console.log('ALARMA')  
} else if (temperature > 70) {  
  console.log('ATENCIÓN')  
}
```

Con 50 °C, este programa **no imprime nada**.

A veces es lo que querés. Casi siempre es un olvido. El `else` te obliga a pensar qué pasa con los casos que no contemplaste.

Un `if` adentro de otro

× ANIDADO

```
if (isEnergized) {  
  if (temperature > 85) {  
    console.log('Alarma')  
  }  
}
```

✓ COMBINADO

```
if (isEnergized &&  
    temperature > 85) {  
  console.log('Alarma')  
}
```

Si un `if` anidado **no tiene** `else`, se puede reemplazar por `&&`.

Cuando la cadena se vuelve repetitiva

```
if (code === 'P') {  
  console.log('Preventivo')  
} else if (code === 'C') {  
  console.log('Correctivo')  
} else if (code === 'D') {  
  console.log('Predictivo')  
}
```

```
switch (code) {  
  case 'P':  
    console.log('Preventivo')  
    break  
  case 'C':  
    console.log('Correctivo')  
    break  
}
```

Misma variable, valores exactos, muchos casos. Ahí `switch` se lee mejor.

Las piezas

PIEZA	QUÉ HACE
<code>switch (variable)</code>	El valor que se va a comparar
<code>case valor:</code>	Si coincide, se ejecuta desde acá
<code>break</code>	Corta y sale del <code>switch</code>
<code>default:</code>	Si no coincidió ningún <code>case</code>

El `default` es el equivalente del `else` : el caso que no contemplaste.

PREGUNTA

Si le sacamos los
break, ¿qué imprime?

El código entra por `case 'P'`. ¿Y después?

Sigue de largo

src/app.js

```
switch (code) {           // code vale 'P'
  case 'P':
    console.log('Preventivo')
  case 'C':
    console.log('Correctivo')
  default:
    console.log('Desconocido')
}
```

SALIDA

```
Preventivo
Correctivo
Desconocido
```

Entró por 'P' y, sin `break`, siguió **sin volver a comparar**.

switch compara con ===

src/app.js

```
const option = prompt('Opción: ') // devuelve '2', no 2

switch (option) {
  case 2:      // nunca coincide: '2' no es 2
  case '2':    // así sí
}
```

ERROR FRECUENTE

Y por eso no sirve para rangos. `case > 85:` no existe. Para rangos, `else if`. Siempre.

if o switch

IF / ELSE IF

Comparás **rangos**

Cada rama pregunta otra cosa

Combinás con `&&` o `||`

Dos o tres casos

SWITCH

Comparás **valores exactos**

Todas miran **la misma** variable

Lista de opciones cerrada

Cuatro o más

En la duda, `if`.

TODO LO QUE HACE `SWITCH` SE PUEDE ESCRIBIR CON `ELSE IF`; AL REVÉS, NO.

PARTE 5

| Comparar lo que se tipea

Si no es si

src/app.js

```
const answer = prompt('¿Pasó? (si/no): ').toLowerCase()

if (answer === 'si') {
  console.log('Aprobada')
} else {
  console.log('Rechazada')
}
```

Sin `.toLowerCase()`, quien escriba `Si` o `SI` entra por el `else`.

PREGUNTA

Leo 100 y 85 con
prompt. ¿Cuál es mayor?

```
if (first > second)
```

Dos textos se comparan como el diccionario

src/app.js

```
const first = prompt('Primera: ')    // '100'  
const second = prompt('Segunda: ')   // '85'  
  
console.log(first > second)
```

SALIDA

false

ERROR FRECUENTE

Compara `'1'` contra `'8'`, letra por letra. **Con `parseFloat` en los dos, da lo correcto.** Y ojo: `'9' > '85'` da `true` — el resultado correcto, por casualidad. Por eso es tan difícil de encontrar.

Todo junto

src/app.js

```
const measured = parseFloat(prompt('Tensión medida [V]: '))
const lowerLimit = NOMINAL_VOLTAGE * (1 - TOLERANCE)
const upperLimit = NOMINAL_VOLTAGE * (1 + TOLERANCE)

if (measured < lowerLimit) {
  console.log('BAJA - revisar la acometida')
} else if (measured <= upperLimit) {
  console.log('NORMAL')
} else {
  console.log('ALTA - riesgo para los equipos')
}
```

Es el ejemplo 07. Correrlo con 300, 372.5 y 405 para ver las tres ramas.

Las tres reglas que más te van a salvar

01  **Los rangos, en orden.** De menor a mayor, sin saltar.

02  **Un `break` por `case`.**

03  **Llaves siempre,** aunque sea una sola línea.

Y la de siempre: `parseFloat` antes de comparar números.

CIERRE

| Corregir código ajeno

La consigna

Un compañero te pasa este programa para que lo mires antes de entregarlo. Verifica si una tensión está dentro de **$\pm 5\%$ sobre 380 V** y decide una acción de mantenimiento.

¿Lo aprobás?

Declaraciones
y nombres

Buenas
prácticas

Decisiones

Las tres cosas que se miran al revisar cualquier código.

El programa

```
ejemplos/09-corregir-el-codigo.js
```

```
git pull
```

Son 32 líneas. Abrilo **entero** en VSCode: para revisar código hay que verlo completo, no de a pedazos.

Copía el contenido a `src/app.js` de tu proyecto y corré `npm run dev`.

Antes de leerlo, corrélo

SALIDA

```
Tensión medida [V]: 300  
Código de servicio: P
```

¿La salida tiene sentido?

Revisar código no es sólo leerlo. Un programa puede estar impecable a la vista y devolver cualquier cosa.

PRÓXIMA CLASE • UNIDAD 05

Bucles

`for` • `while` • `do-while`

Hoy procesás una medición. La próxima, treinta.